Note on 4-Colouring Problem*

Zhongkui Ma

February 19, 2022

1 Algorithm

1.1 Related Concepts

Adjacency in the four-color theorem means that two regions have a common boundary, rather than a finite number of common boundary points; secondly, there are no bizarre regions with the finite area and infinite perimeter in the map (otherwise, the map may not be four-colorable). If there are enclaves on the map, then the map may not be four-colorable. The absence of enclaves on the map means that two non-adjacent regions cannot belong to the same country; that is, there is no restriction that two adjacent regions must have the same color.

In graph theory, the theorem states that for a loopless planar graph G, its chromatic number $\chi(G)$ is not greater than 4, and simply, the graph G is four-colorable. Specifically, there is a simpler statement. A map can be abstracted as an undirected graph G, each region is abstracted as a vertex in the graph, and adjacent regions are abstracted as edges between corresponding vertices in the graph. A graph G is a planar graph, meaning that it is possible to find a way to draw the vertices and edges on a plane such that any two edges do not intersect.

In the following discussion, the default loopless planar graph $G = (V(G), E(G))$ is undirected and unweighted.

The Kempe chain is a chain composed of two-color vertices alternately. First, define a Kempe tree similar to the Kempe chain.

Definition 1 *The **Kempe tree** consists of vertices with two colors, and any two adjacent vertices have different colors.*

Theorem 1 *Any tree has a coloring scheme that can make it a Kempe tree.*

Proof *First, color the tree's root vertex with color 1, then use breadth-first search (BFS) to color the other vertices. If one vertexes' parent vertex is colored with color 1 (or 2), this vertex is colored with color 2 (or 1). Proceed in this way so that the color of each vertex is different from the color of its parent vertex. Because there are no cycles in the tree, each vertex (excluding the root vertex) has only one parent vertex.*

The Kempe chain is a case when the Kempe tree has no branches. A single vertex and Kempe chain are simple Kempe trees.

Definition 2 *For the graph G, the union of the shortest paths from $r \in V(G)$ to all other vertices $i \in V(G)$ obtained by the shortest path algorithm is $\{S(r,i)\}$. The **shortest-path tree** $T(i) = (V(T), E(T))$ is $\cup_{i \in V} S(r,i)$.*

For the shortest-path tree $T(i)$, the edge (v_p, v_q) $(v_p \in T_i(p), v_q \in T_i(q))$ between any two subtrees $T_i(p)$ and $T_i(q)$ formed after removing the root vertex (or a branch vertex) has some special properties.

Theorem 2 *For the shortest-path tree $T(r) = (V(T), E(T))$ of graph G, all subtrees formed by removing the root vertex (or a branch vertex) i are denoted as $\{T_i(j)\}$. There is an edge $(v_p, v_q) \in E(G) - E(T)(v_p \in T_i(p), v_q \in T_i(q))$ between two subtrees. $s_p(v_p)$ is the distance from v_p along the $T_i(p)$ to i, and $s_q(v_q)$ is similarly defined. The equation, $s_p(v_p) - 1 \geq s_q(v_q) \geq s_p(v_p) + 1$, holds.*

*This note does not guarantee the correctness and you can email to me (*zhongkui.ma@uqconnect.edu.au*) if you are interested or have problems with it.*

Proof *$s_p(v_p) - 1 \geq s_q(v_q)$ holds; otherwise, the descendants of v_q will choose v_p as the ancestor vertex in the shortest path algorithm. Similarly, $s_q(v_q) - 1 \geq s_p(v_p)$ holds. Therefore, the equation holds.*

Then define a particular spanning tree, called sloping approximated shortest-path tree.

Theorem 3 *In the graph G, there is such a spanning tree $U(i)$ such that the subtrees $\{U_k(j)\}$ formed by removing the root vertex (or a branch vertex) k have at most one edge between any two subtrees. This tree is called the* ***sloping approximated shortest-path (SASP) tree****.*

Proof *According to theorem 2, for the shortest-path tree, change the parent vertex of v_q to v_p if v_p has child vertices. If the original parent vertex of v_q has other child vertices, then the original parent vertex will become a child vertex of v_q. This procedure is repeated for the parent vertex's parent vertex until the parent vertex no longer has other child vertices. Similar operations are performed for the case where v_q has child vertices. If both vertices have no child vertices, no procedure is required. The same procedure is also performed on each subtree formed by removing branch vertices. Finally, a spanning tree that satisfies the conditions can be obtained.*

And there is the following theorem that is obvious.

Theorem 4 *For a SASP tree, all subtrees formed by removing root vertex or branch vertices, there is only one edge between any two subtrees that do not have a containment relationship. Moreover, at least one of the endpoints of the edge is a leaf vertex of one subtree.*

However, there maybe many edges between two vertices that are in the same subtree.

1.2 Description

Label the four colors in the four-color problem as 1, 2, 3, and 4, and define colors 1 and 2 as dark colors and 3 and 4 as light colors. A Kempe tree composed of two dark colors is called a dark Kempe tree; a light Kempe tree is similarly defined.

The overall idea of the algorithm is to construct a series of Kempe trees and even circles cover in the graph G according to a SASP tree $U(i)$. First, carry out the dark and light coloring. The parent vertex(vertices) and child vertex(vertices) refer to the adjacent vertices in the tree $U(i)$. The algorithm is described below and an example is shown by Figure 1.

1. Set a root vertex $r \in G$, obtain the SASP tree $U(r)$. Paint the root vertex r dark (or light color). Alternately, set the adjacent vertices of the root vertex to dark and light colors.

2. According to the adjacency of the tree $U(r)$, color each vertex i with the following strategy: If the parent vertex of i is dark, then set it to dark color; otherwise, set it to light color. (This process can use depth-first traversal (DFS) or breadth-first traversal (BFS) because here is based on the adjacency of a tree, so the results of both are the same.)

3. According to the adjacency of the tree $U(r)$, each vertex is traversed with the DFS. Moreover, according to the adjacency of the graph G, traverse each adjacent vertex of each vertex. Here the adjacent vertex j that may change its color need to satisfies $s_T(i) = s_T(j)$, where T is the shortest-path tree. Assuming that the vertex i is dark and in a dark subtree (the situation of a light vertex in a light sub tree is similar and symmetric), perform the following color update strategy to void generating a dark odd circle:

   - All adjacent subtrees (including the subtree itself) have at most 1 light-colored subtree, and the others are dark-colored subtrees, then update i to light-colored. If i satisfied the following conditions, it needs not to change color:
     - All adjacent vertices in dark subtrees have been updated or there is no adjacent dark vertex;
     - i has an adjacent dark vertex that has changed color in the same subtree with i.
   - All adjacent subtrees (including the subtree itself) has more than 2 light-colored subtrees and more than 1 dark-colored subtrees, the color of all dark leaf vertices are updated to be light. If i has an adjacent dark vertex in the same subtree with i, change its color to light.

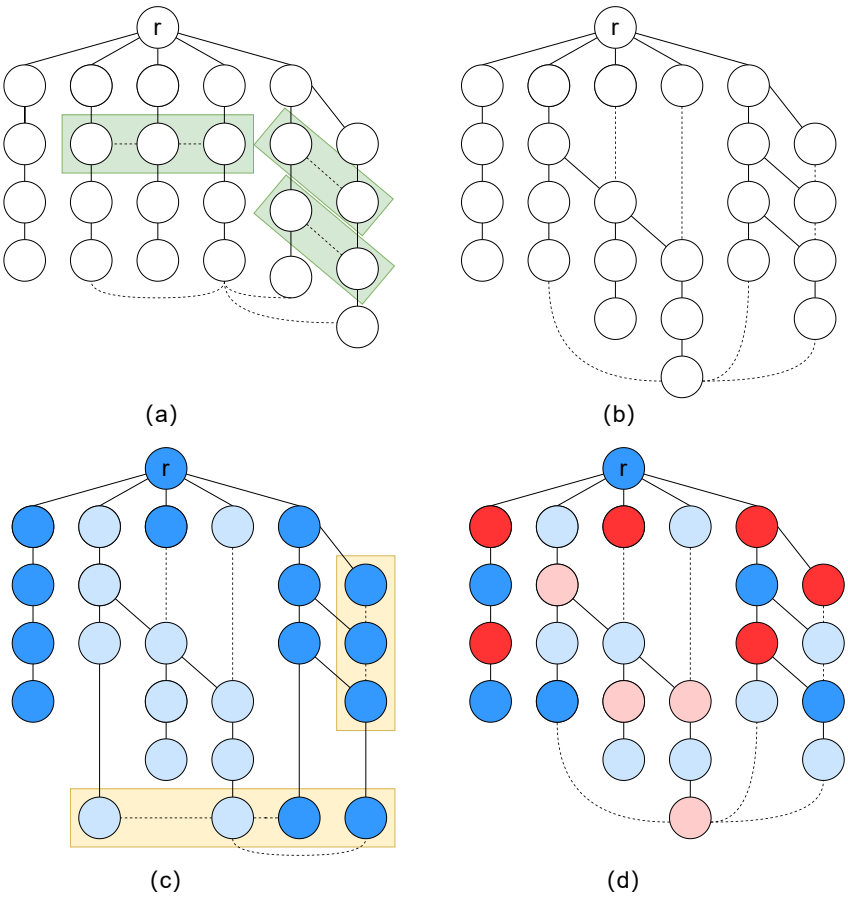

Figure 1: An Example (Figure (a) shows a shortest-path tree. Figure (b) shows a SASP tree from (a). Figure (c) shows the potential dark (or light) circles. Figure (d) shows this loopless planar graph is four-colorable.)

Because the root vertex is dark, for the situation when i is light, there is no need to change the color of i or j if they do not have an ancestor vertex that is a branch a vertex. At this time, it is impossible to generate a light odd circle that contains the dark root vertex.

4. After last step, if one adjacent vertex is in the same subtree of this vertex, perform the following color update strategy to deleting unnecessary color changing (if one vertex has changed color in this procedure, it cannot change twice.):
   - This adjacent vertex j has not been searched that means it is descendant vertex of i (This adjacent vertex must be an ancestor or a descendant of i according to the property of SASP tree.).
     - j has changed color. i needs to change to the initial brightness.
     - Search from this adjacent to i and check if there is a light vertex between them. If so, there is no neeed to change the color of them; otherwise, remain color change.
   - This adjacent vertex j has been searched that means it is ancestor vertex of i. There is nothing to do.
5. Traverse all vertices. If one dark (or light) vertex is unpainted, paint one of dark (or light) color on it, and then perform the BFS for the adjacent dark (or light) vertices, painting them alternately.
6. Determine whether there are two adjacent vertices in the graph G with the same color. If not, the graph is four-colorable; otherwise, it is not.

1.3 Proof

Theorem 5 *After step 3, if there are no two adjacent vertices with the same color, according to the adjacency of the graph G, the adjacent ones with the same brightness compose a tree or even circle. That is, the graph is covered by trees or even circle of different brightness.*

Proof *A series of subtrees $\{T(j)\}$ are generated using step 2, according to the adjacency of $U(r)$. Some edges in G do not in $U(r)$, and two subtrees with the same brightness maybe adjacent according to the adjacency of G. Therefore, these subtrees may not trees and the theorem is not satisfied. In the following description, subtrees refer to $\{T(j)\}$ of different brightness generated in step 2.*

The purpose of step 3 is to split the odd circles among these subtrees to make them trees. Perform DFS on each subtree of $\{U_r(i)\}$, and there is one vertex $v \in U_i(j)$ connected to another branch vertex $w \in U_i(k)$ in different subtree. Because there is a condition of $s_T(i) = s_T(j)$, all vertex may changes color are in one odd circle with one brightness.

Assuming that the current dark vertex v is dark, there will be the following situations if the adjacent subtree is not $U_i(j)$ (light vertices are similar and symmetrical).

- *Case 1: If the adjacent subtrees are all light-colored, they cannot form a circle with the current subtree $U_r(i)$.*
- *Case 2: If the adjacent subtrees are all dark, they may form a dark circle with the current subtree $U_r(i)$. Therefore, the brightness of v needs to be changed.*
- *Case 3: If there is a light-colored subtree and several dark-colored subtrees, the only light-colored subtree cannot form a light-colored circle. Therefore, the brightness of the current vertex needs to be changed to prevent the current subtree $U_r(i)$ and the dark subtrees forming a dark circle.*
- *Case 4: If there are several adjacent subtrees of different brightness, and the number of light-colored subtrees is greater than 2, and the number of dark-colored subtrees is greater than 1, then at most one of the endpoints of these adjacent edges is not a leaf vertex because of the property of SASP tree. The color of leaf vertices is updated to a color different from the brightness of the subtree where it is located.*

When the adjacent vertex of i is in the same subtree with i, the above procedures avoid forming a odd circle with one brightness.

Therefore, if there are no two adjacent vertices with the same color in the graph, the theorem holds after the above procedures.

Theorem 6 *After step 3, if there are two adjacent vertices with the same color, then there is a subgraph that is not 3-colorable.*

Proof *If there are still dark or light circles after performing the above procedures, this only appear in the result of Case 4. After the procedure of Case 4, there are still dark or light circles and there are the following situations:*

- *The two vertices with the same color are in a non-4-colorable subgraph that contains triangles (Figure 2). The only parent vertex of the vertices whose color is adjusted is the root vertex or the same branch vertex v_1 of the SASP tree. Furthermore, there are edges between these leaf vertices, thus forming an odd circle with v_1 and another vertex v_2, which has a complete graph K_t $(t > 4)$ minor.*

- *The two vertices with the same color are in a non-4-colorable triangle-free subgraph. Mycielski graph is an example. In this situation, it means there is still at least one odd circle after step 3 causing two vertices with the same color. This odd dark (or light) circle is consisted of all vertices that has adjusted; otherwise, according to the coloring update rule, the circle will be destroyed. All vertices of the odd circle cannot be changed because it will cause another odd dark (or light) circle generating.*

Therefore, this structure requires at least five colors to be painted, which is not four-colorable.

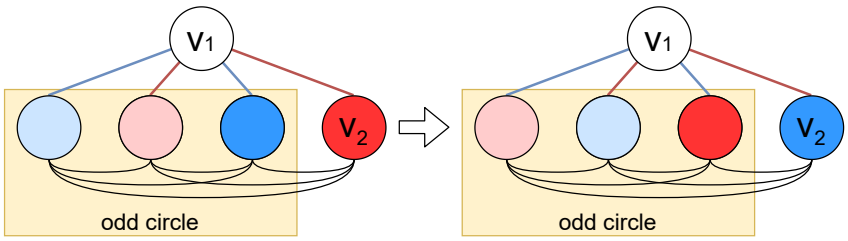

Figure 2: A Counterexample with a K_5 minor (No matter which color the leaf vertices is painted, the graph is not four-colorable. And at this time, the graph is not a planar graph because there is a intersection of two edges.)

Theorem 7 *The above algorithm can determine whether a graph is four-colorable.*

Proof *Because the brightness of adjacent trees is different after step 3, each two vertices of adjacent trees have different colors. Because the tree must have a coloring scheme to make it a Kempe tree, each two adjacent vertices in the tree have different color. The colors between any two adjacent subtrees are also different. Therefore, there are only four colors in total.*

If a graph cannot meet the four-coloring requirement under the above algorithm, then it is not 4-colorable.

1.4 Complexity

Suppose the graph G has m edges and n vertices. Use an adjacency mapp to store the graph. The time comlexity of each step is as follows:

- The time complexity of step 1 is $O((m+n)\log n)$. Because the time complexity required to generate the shortest-path tree is $O((m+n)\log n)$, the time complexity required to build a SASP tree is $O(m+n)$.

- The time complexity of step 3 is $O(n^2)$ because it needs to traverse all vertices and adjacent vertices of each vertex.

- The time complexity of step 2, 4 and 5 is $O(m+n)$ because it needs to traverse all edges and vertices.

Therefore, the total time complexity is $O(m+n^2)$.